BATCH 4

Tool-by-Tool Integration Map

Dragonfruit Media × AIGenX

Every tool in the DFM stack — how agents connect, what they automate, and the integration architecture behind each connection.

PART 1 OF 2 — CORE PLATFORM TOOLS

March 2026 | Confidential

Current Monthly Tool Spend: $12,296/mo ($147,555/yr)

Integration-Critical Tools: 12 | Agent Touchpoints: 47

Table of Contents — Part 1

1 Integration Architecture Overview

2 Integration Method Decision Framework

3 Current Tool Spend Analysis

4 Tool 1: ClickUp — $2,114/mo, 104 users

5 Tool 2: Slack — $866/mo, 99 users

6 Tool 3: Notion — $1,176/mo, 98 users

7 Tool 4: Frame.io — $1,095/mo, 10 users

8 Tool 5: Dropbox — $768/mo

9 Tool 6: Make — $560/mo

1. Integration Architecture Overview

Every agent in the AIGenX system connects to DFM's tool stack through one of five integration methods. The choice of method for each tool is driven by three factors: **latency requirements**, **data sensitivity**, and **action complexity**.

Five Integration Methods

Method	Badge	Description	Latency	Best For
MCP Server	MCP	Model Context Protocol — standardized tool interface. Agent calls tools through a unified schema. Server handles auth, rate limiting, and response formatting.	200-500ms	ClickUp, Slack, Notion — tools with official MCP servers
REST API (Function Tool)	API	Direct API calls wrapped as OpenAI Agents SDK function tools. Agent receives typed parameters, function executes API call, returns structured response.	100-800ms	Frame.io, YouTube, Dropbox — tools without MCP but with robust APIs
Webhook Listener	WEBHOOK	Event-driven triggers. External tool fires webhook → our listener parses event → routes to appropriate agent for action.	Event-driven	ClickUp status changes, Frame.io comments, Slack events
Make/Zapier Scenario	MAKE	Low-code automation for multi-step workflows that don't require agent reasoning. Scheduled or triggered. Existing DFM Make scenarios preserved.	1-30s	File transfers, scheduled reports, data sync between tools
Native Integration	NATIVE	Built-in integrations between tools (e.g., Frame.io ↔ Adobe Premiere, Descript ↔ Dropbox). No agent involvement — preserved as-is.	Varies	Editor workflows, creative tool chains

Integration Layer Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                     AGENT ORCHESTRATOR                       │
│               (OpenAI Agents SDK — Runner)                   │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐   │
│   │ Producer │   │    PC    │   │   PPM    │   │    CD    │   │
│   │  Agent   │   │  Agent   │   │  Agent   │   │  Agent   │   │
│   └────┬─────┘   └────┬─────┘   └────┬─────┘   └────┬─────┘   │
│        │              │              │              │         │
├────────┴──────────────┴──────────────┴──────────────┴─────────┤
│                     TOOL BINDING LAYER                       │
│                                                              │
```

```
│  ┌──────────────────────────────────────────────────────┐    │
│  │              MCP Client (mcp library)                 │    │
│  │  ┌──────────┐  ┌──────────┐  ┌──────────┐            │    │
│  │  │ClickUp   │  │  Slack   │  │ Notion   │            │    │
│  │  │MCP Srv   │  │MCP Srv   │  │MCP Srv   │            │    │
│  │  └──────────┘  └──────────┘  └──────────┘            │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                               │
│  ┌──────────────────────────────────────────────────────┐    │
│  │           Function Tools (REST API wrappers)          │    │
│  │  ┌──────────┐  ┌──────────┐  ┌──────────┐            │    │
│  │  │Frame.io  │  │YouTube   │  │Dropbox   │            │    │
│  │  │  API     │  │Data API  │  │  API     │            │    │
│  │  └──────────┘  └──────────┘  └──────────┘            │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                               │
│  ┌──────────────────────────────────────────────────────┐    │
│  │           Webhook Listeners (FastAPI)                 │    │
│  │  POST /webhooks/clickup                               │    │
│  │  POST /webhooks/frameio                               │    │
│  │  POST /webhooks/slack/events                          │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                               │
│  ┌──────────────────────────────────────────────────────┐    │
│  │           Make/Zapier (Preserved + Extended)          │    │
│  │  Existing: 6 Make scenarios, 1 Zapier zap             │    │
│  │  New: Agent-triggered Make webhooks                   │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

2. Integration Method Decision Framework

For each tool in DFM's stack, we apply the following decision tree to determine the optimal integration method:

```
START
  |
  ├─ Does the tool have an official MCP server?
  |    ├─ YES → Does the MCP server cover all needed operations?
  |    |         ├─ YES → Use MCP exclusively
  |    |         └─ NO  → Use MCP + Function Tools for gaps
  |    |
  |    └─ NO → Does the tool have a REST API?
  |           ├─ YES → Does the agent need to reason about the response?
  |           |         ├─ YES → Wrap as Function Tool
  |           |         └─ NO  → Can Make/Zapier handle it?
  |           |                   ├─ YES → Use Make/Zapier
  |           |                   └─ NO  → Wrap as Function Tool
  |           |
  |           └─ NO → Does the tool support webhooks?
  |                   ├─ YES → Webhook listener + agent routing
  |                   └─ NO  → Manual / Native integration only
```

Decision Matrix — All DFM Tools

Tool	MCP?	REST API?	Webhooks?	Primary Method	Secondary
ClickUp	✅ Official	✅ v2	✅	MCP	WEBHOOK
Slack	✅ Official	✅ Web API	✅ Events API	MCP	WEBHOOK
Notion	✅ Official	✅ v1	❌	MCP	API
Frame.io	❌	✅ v2	✅	API	WEBHOOK
Dropbox	❌	✅ v2	✅	API	MAKE
YouTube Data API	❌	✅ v3	❌ (PubSubHubbub)	API	MAKE
Make	❌	✅ Webhook triggers	✅	MAKE	WEBHOOK
Zapier	❌	✅ Webhook triggers	✅	MAKE	—
Descript	❌	⚠️ Limited	❌	NATIVE	MAKE
Adobe Suite	❌	⚠️ CEP/UXP	❌	NATIVE	—
		✅	✅	API	MCP

Google Workspace	✅ Community				
Zoom	❌	✅ v2	✅	API	MAKE
Nusii	❌	✅	✅	WEBHOOK	MAKE
QuickBooks	❌	✅	✅	API	MAKE
HelloSign	❌	✅	✅	API	WEBHOOK

3. Current Tool Spend Analysis

$12,296/mo

Total Active Monthly Tool Spend ($147,555/yr)

Spend by Integration Criticality

Tier	Tools	Monthly Spend	% of Total	Agent Integration
Tier 1: Core Pipeline *Agent-integrated*	ClickUp, Slack, Notion, Frame.io, Dropbox	$6,019/mo	48.9%	MCP API WEBHOOK
Tier 2: Automation *Agent-orchestrated*	Make, Zapier	$693/mo	5.6%	MAKE WEBHOOK
Tier 3: Creative *Agent-adjacent*	Adobe Suite, Descript, Storyblocks, Artlist, Epidemic, Misterhorse, ElevenLabs, Freepik	$1,504/mo	12.2%	NATIVE
Tier 4: Communication *Agent-enhanced*	Zoom, Missive, Grain, Loom	$1,598/mo	13.0%	API MAKE
Tier 5: Operations *Agent-connected*	Google Workspace, LastPass, QBO, Nusii, HelloSign, Webflow	$1,661/mo	13.5%	API WEBHOOK
Tier 6: Analytics *Agent-consumed*	1of10, ViewStats, YouTube	$212/mo	1.7%	API

Key Insight: 48.9% of tool spend ($6,019/mo) flows through Tier 1 tools that agents will directly integrate with via MCP and API. These 5 tools are touched by every role and every pipeline phase. Maximizing agent coverage here delivers the highest ROI per integration dollar.

4. Tool 1: ClickUp

Current Spend	$2,113.99/mo ($25,368/yr) — Business Plus Plan
Users	104 members (all roles)
Pipeline Phases	Onboarding, Pre-Production, Post-Production
Integration Method	MCP Primary + WEBHOOK Event-driven + MAKE Existing scenarios
MCP Server	Official ClickUp MCP Server (confirmed available)
Time Impact	Producers: 38% → 18% | PCs: 40% → 20% | PPMs: 5% → 1% | CDs: 1.7% → 0% | Editors: 2.5% → 1%

4.1 Why ClickUp Is the #1 Integration Priority

ClickUp administration consumes more team hours than any other single activity across DFM. The time audit data is unambiguous:

- **Producers:** 38% of capacity (93 hours/week across 6 people) on ClickUp admin
- **PCs:** 40% of capacity (127 hours/week across 8 people) on ClickUp admin
- **Combined:** 220 hours/week of human time on task creation, status updates, date management, assignees, comments, and space management
- **Dream target:** 110 hours/week — a 50% reduction via automation

4.2 MCP Server Capabilities

The official ClickUp MCP server exposes the following tool categories that our agents will consume:

MCP Tool Category	Operations Available	Agent Consumer	Pipeline Step
Spaces	create_space, get_spaces, get_space	Producer Agent	Onboarding — Create Template Client Space
Folders	create_folder, get_folders	Producer Agent	Pre-Production — Apply template batch
Lists	create_list, get_lists	Producer Agent, PC Agent	Pre-Production — Batch setup
Tasks	create_task, update_task, get_tasks, get_task, delete_task	All agents	All phases — task CRUD
Comments	create_comment, get_comments	PC Agent, PPM Agent	Post-Production — feedback routing
Members	get_workspace_members	PC Agent (ED-2)	Post-Production — capacity-based assignment

Custom Fields	get_custom_fields, set_custom_field	Producer Agent, PC Agent	Pre-Production — dates, assignees, metadata
Tags	get_tags, add_tag	Producer Agent	Pre-Production — content type tagging
Dependencies	add_dependency, delete_dependency	Producer Agent (PR-1)	Pre-Production — timeline dependencies

4.3 Agent-to-ClickUp Integration Architecture

```
# ClickUp MCP Server Configuration
# File: config/clickup_mcp.json
{
  "server": "clickup-mcp-server",
  "version": "1.0.0",
  "auth": {
    "type": "oauth2",
    "token_env": "CLICKUP_API_TOKEN",
    "workspace_id_env": "CLICKUP_WORKSPACE_ID"
  },
  "rate_limits": {
    "requests_per_minute": 100,
    "burst": 10
  },
  "tool_permissions": {
    "producer_agent": [
      "create_space", "get_spaces", "create_folder", "get_folders",
      "create_list", "get_lists", "create_task", "update_task",
      "get_tasks", "add_dependency", "set_custom_field", "add_tag"
    ],
    "pc_agent": [
      "get_tasks", "update_task", "create_comment", "get_comments",
      "get_workspace_members", "set_custom_field"
    ],
    "ppm_agent": [
      "get_tasks", "update_task", "create_comment", "get_comments"
    ],
    "cd_agent": [
      "get_tasks", "update_task"
    ],
    "editor_support_agent": [
      "get_tasks", "update_task"
    ]
  }
}
```

4.4 Webhook Event Handlers

ClickUp webhooks fire on task events. Our FastAPI listener routes these to the appropriate agent:

```python
# Webhook handler for ClickUp events
# File: webhooks/clickup_handler.py

from fastapi import APIRouter, Request
from agents import Runner
from config import get_agent_by_event_type

router = APIRouter()

CLICKUP_EVENT_ROUTING = {
    "taskStatusUpdated": {
        "agent": "pc_agent",
        "action": "handle_status_change",
        "conditions": {
            "status_to:ready_to_start": "notify_editor_assignment",
            "status_to:in_review": "trigger_ppm_review",
            "status_to:approved": "trigger_publish_prep"
        }
    },
    "taskCreated": {
        "agent": "pc_agent",
        "action": "validate_and_enrich_task",
        "auto_actions": [
            "check_required_fields",
            "set_default_dates_if_missing",
            "assign_based_on_capacity"
        ]
    },
    "taskCommentPosted": {
        "agent": "ppm_agent",
        "action": "route_feedback",
        "conditions": {
            "comment_contains:@editor": "forward_to_editor_slack",
            "comment_contains:revision": "create_revision_subtask"
        }
    },
    "taskDueDateUpdated": {
        "agent": "producer_agent",
        "action": "cascade_timeline_update",
        "auto_actions": [
            "recalculate_dependent_dates",
            "notify_affected_team_members",
            "check_for_conflicts"
        ]
    }
}

@router.post("/webhooks/clickup")
async def handle_clickup_webhook(request: Request):
    payload = await request.json()
    event_type = payload.get("event")

    routing = CLICKUP_EVENT_ROUTING.get(event_type)
    if not routing:
        return {"status": "ignored", "event": event_type}
```

```
    agent = get_agent_by_event_type(routing["agent"])
    context = build_context_from_payload(payload)

    # HITL gate check
    if requires_human_approval(event_type, payload):
        await send_approval_request(context)
        return {"status": "pending_approval"}

    result = await Runner.run(agent, input=context)
    return {"status": "processed", "result": result.final_output}
```

4.5 Existing Make Scenarios — Preserved & Extended

DFM currently runs 6 Make scenarios connected to ClickUp. All are preserved; 3 are extended with agent triggers:

Existing Make Scenario	Current Function	Status	Agent Extension
Production Timeline Automation	Triggers on new parent tasks	✅ Preserved	Producer Agent now sets trigger data more accurately via MCP
Creation of Subtasks in Editing Batches	Auto-names subtasks	✅ Preserved	PC Agent validates subtask completeness post-creation
Subtask Status → Webhook	Fires when status = "ready to start" for Video/Short tasks	✅ Preserved	PC Agent (ED-2) intercepts to run capacity-based assignment
Auto-Link Editor via Relationship	Links editor to Editor Database on V1 assignment	✅ Preserved	PPM Agent reads this link for performance tracking
Weekly Accumulator (×3)	Various editor database tracking steps	✅ Preserved	PPM Agent consumes accumulated data for pattern reports
Missed Due Date Tracker (Zapier)	Tracks missed subtask due dates	✅ Preserved	Producer Agent receives alerts and auto-cascades timeline adjustments

4.6 Pipeline Steps Automated via ClickUp Integration

Phase	Pipeline Step	Current (Manual)	Automated (Agent)	Method
Onboarding	Create Template Client Space	Producer manually creates space, folders, lists	Producer Agent: "Set up [Client] space" → MCP creates full structure from template	MCP
Onboarding	Fill Out ClickUp Space	Producer manually enters client info across fields	Agent pulls from KO Survey + Notion portal → populates all custom fields	MCP
Pre-Prod	Apply template batch	Producer applies batch template, customizes	Agent applies template + auto-customizes based on client config	MCP
Pre-Prod	Set publish dates			MCP

		Producer manually sets dates	PR-1 Timeline Calculator → MCP sets all dates with dependencies	
Pre-Prod	Set due dates (backtrack)	Producer manually calculates backwards from publish	Agent auto-calculates: publish date – buffer – review cycles – edit time	MCP
Pre-Prod	Assign tasks	Producer/PC manually checks availability, assigns	ED-2 agent: capacity check → skill match → auto-assign with HITL confirmation	MCP
Post-Prod	Editor uploads & tags CD	Editor manually uploads, tags in ClickUp	Webhook detects upload → PC Agent auto-notifies CD via Slack	WEBHOOK
Post-Prod	Status updates (V1→V2→V3→V4)	Manual status changes by editors, PCs	Webhook chain: status change → notify next reviewer → update timeline	WEBHOOK
Post-Prod	Publish prep tracking	YPC manually updates tracker, notifies publisher	Agent detects "approved" status → auto-generates publish prep checklist	MCP WEBHOOK

ClickUp Integration ROI: 220 hours/week currently spent on ClickUp admin → target 110 hours/week. At an average blended rate of $25/hr, this represents **$143,000/year in recovered capacity** — 5.6× the annual ClickUp subscription cost.

5. Tool 2: Slack

Current Spend	$866.25/mo ($10,395/yr) — Pro Plan
Users	99 members (all roles)
Pipeline Phases	Pre-Production, Post-Production, Onboarding
Integration Method	MCP Primary + WEBHOOK Events API
MCP Server	Official Slack MCP Server (confirmed available)
Time Impact	Producers: 17% → 10% | PCs: 11% → 10% | PPMs: 14% → 3% | CDs: 8% → 5%

5.1 Slack's Role in the DFM Pipeline

Slack serves three distinct functions at DFM, each requiring different agent integration patterns:

Function	Current Usage	Agent Automation
Internal Communication	Team coordination, status updates, questions, standup discussions	Async standup bot, auto-generated agendas, notification routing
Client Communication	External Slack channels per client — feedback, approvals, deliverable sharing	Template-based messages (PR-3), sentiment monitoring (PR-2), auto-routing to ClickUp
Agent Interface	N/A (new)	Slash commands for agent interaction: /create-task, /timeline, /roast, /assign

5.2 MCP Server Capabilities

```
# Slack MCP Server — Available Tools
{
  "tools": [
    {
      "name": "send_message",
      "description": "Send a message to a Slack channel or DM",
      "params": ["channel_id", "text", "blocks", "thread_ts"]
    },
    {
      "name": "list_channels",
      "description": "List channels in the workspace",
      "params": ["types", "limit"]
    },
    {
      "name": "get_channel_history",
      "description": "Retrieve message history from a channel",
      "params": ["channel_id", "limit", "oldest", "latest"]
    },
```

```
    {
      "name": "get_thread_replies",
      "description": "Get replies in a message thread",
      "params": ["channel_id", "thread_ts"]
    },
    {
      "name": "search_messages",
      "description": "Search for messages across channels",
      "params": ["query", "sort", "count"]
    },
    {
      "name": "get_users",
      "description": "List workspace members",
      "params": ["limit", "cursor"]
    },
    {
      "name": "get_user_profile",
      "description": "Get a user's profile information",
      "params": ["user_id"]
    },
    {
      "name": "add_reaction",
      "description": "Add an emoji reaction to a message",
      "params": ["channel_id", "timestamp", "name"]
    }
  ]
}
```

5.3 Slack Bot Architecture — Agent Gateway

The Slack bot serves as the primary human-to-agent interface. Team members interact with agents through natural language in Slack, and agents respond through Slack notifications.

```
# Slack Bot — Agent Gateway
# File: integrations/slack_bot.py

SLASH_COMMANDS = {
    "/create-task": {
        "agent": "producer_agent",
        "description": "Create a ClickUp task from natural language",
        "example": "/create-task Mango Pod longform edit for [Client], due Friday,
assign to [Editor]",
        "hitl_required": True,
        "confirmation_channel": "#agent-actions"
    },
    "/timeline": {
        "agent": "producer_agent",
        "description": "Calculate project timeline",
        "example": "/timeline 4 longform + 8 shorts, client review 48h, pod capacity
normal",
        "hitl_required": False
    },
    "/assign": {
        "agent": "pc_agent",
```

```
        "description": "Auto-assign editor based on capacity",
        "example": "/assign longform edit task-id-123",
        "hitl_required": True,
        "confirmation_channel": "#agent-actions"
    },
    "/roast": {
        "agent": "cd_agent",
        "description": "Get thumbnail/packaging critique",
        "example": "/roast [attach thumbnail image + title]",
        "hitl_required": False
    },
    "/qc": {
        "agent": "ppm_agent",
        "description": "Run automated QC check on video",
        "example": "/qc frame-io-link client-name",
        "hitl_required": False
    },
    "/sentiment": {
        "agent": "ppm_agent",
        "description": "Get client sentiment summary",
        "example": "/sentiment [client-name] last-30-days",
        "hitl_required": False
    },
    "/standup": {
        "agent": "pc_agent",
        "description": "Generate async standup from ClickUp data",
        "example": "/standup mango-pod",
        "hitl_required": False
    }
}
```

5.4 Notification Routing Architecture

Agents send notifications to specific Slack channels based on event type and audience:

Channel Pattern	Purpose	Agent Source	Message Type
`#agent-actions`	All agent actions requiring HITL approval	All agents	Interactive buttons: ✅ Approve / ❌ Reject / ✏️ Modify
`#[pod]-production`	Pod-specific production updates	PC Agent, Producer Agent	Task assignments, status changes, deadline alerts
`#[client]-external`	Client-facing communication	Producer Agent (PR-3)	Templated updates, deliverable notifications
`#editorial-alerts`	Editor assignments and feedback	PC Agent, PPM Agent	New assignments, revision requests, QC results
`#creative-review`	Creative review notifications	CD Agent	Roadmap ready, thumbnail feedback, packaging scores
`#daily-digest`	Morning standup summaries	PC Agent	Auto-generated from ClickUp: what shipped, what's blocked, what's due

5.5 Slack-to-ClickUp Routing (Automated)

One of the highest-impact automations: messages in Slack that contain actionable items are automatically routed to ClickUp tasks.

```
# Slack → ClickUp routing logic
# File: integrations/slack_to_clickup_router.py

ROUTING_PATTERNS = [
    {
        "pattern": r"(?:can you|please|need to)\s+(?:create|make|add)\s+(?:a\s+)?
task",
        "action": "create_clickup_task",
        "agent": "producer_agent",
        "extract": ["task_description", "assignee", "due_date", "pod"]
    },
    {
        "pattern": r"(?:deadline|due date)\s+(?:changed?|moved?|pushed?|extended?)",
        "action": "update_clickup_dates",
        "agent": "producer_agent",
        "extract": ["task_reference", "new_date", "reason"]
    },
    {
        "pattern": r"(?:v[1-4]|edit|revision)\s+(?:is\s+)?(?:ready|done|complete|
uploaded)",
        "action": "update_clickup_status",
        "agent": "pc_agent",
        "extract": ["task_reference", "version", "link"]
    },
    {
        "pattern": r"(?:approved?|lgtm|looks good|ship it)",
        "action": "advance_clickup_status",
        "agent": "pc_agent",
        "extract": ["task_reference", "approver"]
    }
]

# Agent processes the routed message with full context
async def route_slack_to_clickup(message, channel, user):
    for rule in ROUTING_PATTERNS:
        if re.search(rule["pattern"], message, re.IGNORECASE):
            context = extract_entities(message, rule["extract"])
            context["source_channel"] = channel
            context["source_user"] = user

            agent = get_agent(rule["agent"])
            result = await Runner.run(agent, input=json.dumps({
                "action": rule["action"],
                "context": context,
                "original_message": message
            }))

            # Post confirmation back to Slack
            await slack_mcp.send_message(
                channel=channel,
```

```
            text=f"✅ {result.final_output}",
            thread_ts=message.get("ts")
        )
        return result
```

> **Slack Integration ROI:** Combined Slack communication time across all roles: ~135 hours/week → target ~80 hours/week. The Slack bot becomes the primary agent interface, meaning team members don't need to learn new tools — they interact with AI through the tool they already use most.

6. Tool 3: Notion

Current Spend	$1,176/mo ($14,112/yr) — Plus Plan
Users	98 members (all roles)
Pipeline Phases	Onboarding, Pre-Production, Post-Production, Production
Integration Method	MCP Primary + API for database queries
MCP Server	Official Notion MCP Server (confirmed available)

6.1 Notion's Role in the DFM Pipeline

Notion serves as DFM's knowledge layer — client portals, ideation docs, scripts, SOPs, style guides, and channel audits all live here. Unlike ClickUp (task management) and Slack (communication), Notion is the **content and knowledge repository**.

Notion Function	Content Type	Pipeline Phase	Agent Interaction
Client Portals	Client info, calendars, brand docs	Onboarding	CD Agent reads for CD-4 Knowledge Bases; Producer Agent populates during onboarding
Ideation Docs	Video concepts, research, brainstorms	Pre-Production	CD Agent writes ideation drafts; CD-1 Retention Scorer reads scripts from here
Scripts	V1/V2/V3 script drafts	Pre-Production	CD Agent reads for voice matching (CD-5); Editor reads for edit prep
Style Guides / SOPs	Editing SOPs, style briefs, brand guidelines	Onboarding / Post-Prod	PPM Agent reads for QC checklist generation (ED-1); CD Agent reads for style context
Channel Audits	Analytics, competitor analysis, strategy docs	Onboarding	CD-2 Competitor Scraper writes weekly digests here
Monthly Reports	Performance briefs, DA analysis	Post-Production	CD Agent reads for ideation context; auto-populated by analytics pipeline
Packaging Docs	Title, thumbnail, description, tags	Post-Production	CD Agent writes final title/thumb selections; Publisher reads for upload

6.2 MCP Server Operations

```
# Notion MCP Server — Key Operations Used by Agents
{
  "tools": [
    {
      "name": "search",
      "description": "Search across all Notion pages and databases",
      "agent_use": "CD Agent searches for client-specific content, past scripts,
```

```
brand docs"
    },
    {
      "name": "get_page",
      "description": "Retrieve a specific Notion page by ID",
      "agent_use": "All agents retrieve specific docs (scripts, SOPs, ideation)"
    },
    {
      "name": "create_page",
      "description": "Create a new page in a database or as a child",
      "agent_use": "CD Agent creates ideation docs; CD-2 creates weekly competitor
digests"
    },
    {
      "name": "update_page",
      "description": "Update page properties",
      "agent_use": "Producer Agent updates client portal; CD Agent updates packaging
docs"
    },
    {
      "name": "get_database",
      "description": "Retrieve database schema and entries",
      "agent_use": "PPM Agent queries Editor Database for performance data"
    },
    {
      "name": "query_database",
      "description": "Query a database with filters and sorts",
      "agent_use": "CD-4 Knowledge Base queries client-specific databases"
    },
    {
      "name": "append_block_children",
      "description": "Append content blocks to a page",
      "agent_use": "CD Agent appends research findings; PPM Agent appends QC
reports"
    }
  ]
}
```

6.3 Client Knowledge Base Architecture (CD-4)

The CD-4 tool creates per-client AI knowledge bases grounded in Notion content. Here's how the Notion MCP enables this:

```
# CD-4: Client Knowledge Base — Notion Integration
# File: agents/cd_tools/knowledge_base.py

class ClientKnowledgeBase:
    """
    Per-client knowledge base built from Notion content.
    CD can query conversationally for on-brand suggestions.
    """

    def __init__(self, client_name: str, notion_mcp):
        self.client = client_name
```

```
        self.notion = notion_mcp
        self.index = None  # Vector index built from Notion pages

    async def build_index(self):
        """Crawl all Notion pages for this client and build searchable index"""

        # 1. Find client portal page
        portal = await self.notion.search(
            query=f"{self.client} Portal",
            filter={"object": "page"}
        )

        # 2. Crawl all child pages recursively
        pages = await self.crawl_children(portal.results[0].id)

        # 3. Extract content categories
        content = {
            "brand_voice": [],      # Style guides, tone docs
            "past_scripts": [],     # All previous scripts
            "performance_data": [], # Monthly reports, analytics
            "audience_info": [],    # Demographics, preferences
            "competitor_intel": [], # Channel audits, competitor analysis
            "ideation_history": []  # Past ideation docs
        }

        for page in pages:
            category = self.classify_page(page)
            content[category].append(page.content)

        # 4. Build vector index for semantic search
        self.index = build_vector_index(content)

        return {
            "client": self.client,
            "pages_indexed": len(pages),
            "categories": {k: len(v) for k, v in content.items()},
            "last_updated": datetime.now().isoformat()
        }

    async def query(self, question: str) -> str:
        """CD asks a question, gets client-grounded answer"""

        # Semantic search across all client content
        relevant_chunks = self.index.search(question, top_k=10)

        # Build context-aware prompt
        context = "\n\n".join([
            f"[{chunk.category}] {chunk.text}"
            for chunk in relevant_chunks
        ])

        return f"""Based on {self.client}'s brand documentation:

{context}

Answer the CD's question: {question}
```

```
Ground your response in the client's actual voice, past decisions,
and performance data. Cite specific documents where relevant."""
```

6.4 Pipeline Steps Automated via Notion Integration

Phase	Step